

# CloudRoute Lab - Complete Infrastructure Blueprint

---

**Project:** CloudRoute Lab  
**Type:** Student Kubernetes Lab (Cost-Aware)  
**Architecture:** Self-managed Kubernetes on AWS EC2  
**Nodes:** 1 Master (Control Plane) + 1 Worker

## 1. Cloud Network Structure (AWS VPC)

---

### VPC Configuration

```
VPC Name:      cloudroute-vpc
VPC CIDR:      10.0.0.0/16
Region:        us-east-1 (or your preferred region)
Tenancy:        Default (shared)
DNS Hostnames: Enabled
DNS Resolution: Enabled
```

### Subnet Configuration

```
Public Subnet Name:  cloudroute-public-subnet-1a
Public Subnet CIDR:  10.0.1.0/24
Availability Zone:    us-east-1a
Auto-assign Public IP: Yes
```

### Internet Gateway

```
IGW Name:    cloudroute-igw
Attached:    cloudroute-vpc
```

## Route Table

```
Route Table Name:    cloudroute-public-rt
Association:         cloudroute-public-subnet-1a
Routes:
  - 0.0.0.0/0 → cloudroute-igw (Internet Gateway)
```

## Security Groups

### Master Node Security Group

```
SG Name:            cloudroute-master-sg
Inbound Rules:
  - Port 22 (SSH)      → 0.0.0.0/0 (or your IP only)
  - Port 6443 (API)    → 0.0.0.0/0 (or worker SG only)
  - Port 2379-2380 (etcd) → Self (cloudroute-master-sg)
  - Port 10250 (kubelet) → Self + Worker SG
  - Port 10259 (scheduler) → Self
  - Port 10257 (controller) → Self
  - Port 80 (HTTP)     → 0.0.0.0/0
  - Port 443 (HTTPS)   → 0.0.0.0/0
  - Port 30000-32767 (NodePort) → 0.0.0.0/0
Outbound Rules:
  - All traffic → 0.0.0.0/0
```

### Worker Node Security Group

```
SG Name:            cloudroute-worker-sg
Inbound Rules:
  - Port 22 (SSH)      → 0.0.0.0/0 (or your IP only)
  - Port 10250 (kubelet) → Master SG only
  - Port 30000-32767 (NodePort) → 0.0.0.0/0
  - Port 80 (HTTP)     → 0.0.0.0/0
  - Port 443 (HTTPS)   → 0.0.0.0/0
```

## Outbound Rules:

- All traffic → 0.0.0.0/0

## 2. Compute Structure (AWS EC2)

### Master Node (Control Plane)

```
Instance Name:      cloudroute-master
Instance Type:      t3.medium (2 vCPU, 4 GB RAM)
AMI:               Ubuntu 22.04 LTS (HVM, EBS-backed)
Volume Type:       gp3
Volume Size:       20 GB
Volume Name:       cloudroute-master-root
Key Pair:          cloudroute-keypair
Security Group:    cloudroute-master-sg
Subnet:            cloudroute-public-subnet-1a
Public IP:         Yes (or Elastic IP)
IAM Role:          cloudroute-master-role
Hostname:          cloudroute-master
Tags:
  Name: cloudroute-master
  Role: master
  Project: cloudroute-lab
```

### Worker Node

```
Instance Name:      cloudroute-worker
Instance Type:      t3.medium (2 vCPU, 4 GB RAM)
AMI:               Ubuntu 22.04 LTS (HVM, EBS-backed)
Volume Type:       gp3
Volume Size:       20 GB
Volume Name:       cloudroute-worker-root
Key Pair:          cloudroute-keypair
Security Group:    cloudroute-worker-sg
Subnet:            cloudroute-public-subnet-1a
Public IP:         Yes
IAM Role:          cloudroute-worker-role
Hostname:          cloudroute-worker
Tags:
  Name: cloudroute-worker
```

```
Role: worker
Project: cloudroute-lab
```

## SSH Key Pair

```
Key Pair Name:    cloudroute-keypair
Key Type:         RSA 2048-bit
Private Key:      Save locally as cloudroute-keypair.pem
File Permissions: chmod 400 cloudroute-keypair.pem
```

## Instance Type Justification

```
t3.medium:  - 2 vCPU, 4 GB RAM
              - Burstable performance (good for labs)
              - ~$30/month per instance (on-demand)
              - Total: ~$60/month for both nodes
              - Sufficient for kubeadm + small workloads
```

# 3. Kubernetes Cluster Plan

---

## Cluster Configuration

```
Cluster Name:      cloudroute-cluster
Kubernetes Version: v1.29.x (latest stable)
Container Runtime: containerd (recommended)
CNI Plugin:        Calico (recommended for NetworkPolicy support)
Pod CIDR:          10.244.0.0/16
Service CIDR:      10.96.0.0/12
DNS:               CoreDNS
```

## Master Node Role

```
Components:
- kube-apiserver
```

- kube-scheduler
- kube-controller-manager
- etcd (single-node)
- kubelet
- kube-proxy
- Calico node

Taints:

- node-role.kubernetes.io/master:NoSchedule  
(Prevent regular pods from running on master)

## Worker Node Role

Components:

- kubelet
- kube-proxy
- Calico node

Labels:

- node-role.kubernetes.io/worker: ""

## Container Runtime Setup

```
Runtime:          containerd
Cgroup Driver:    systemd
Sandbox Image:    registry.k8s.io/pause:3.9
```

## kubeadm Initialization

Control Plane Endpoint: :6443  
(or use internal IP if within VPC)

kubeconfig Output:

- /etc/kubernetes/admin.conf → copy to ~/.kube/config on master
- Or output to: ~/cloudroute-kubeconfig

## CNI Plugin (Calico)

```
Installation:      kubectl apply -f https://raw.githubusercontent.com/projectcalico/
Pod CIDR:          10.244.0.0/16
Features:
  - NetworkPolicy support
  - Network isolation
  - Good for learning K8s networking
```

## 4. Load Balancing and Networking Plan

### Service Types and Usage

#### ClusterIP Services

```
Frontend ClusterIP:
  Name:      cloudroute-frontend
  Type:      ClusterIP
  Port:      80 → TargetPort 3000
  Selector:   app=cloudroute-frontend
  Usage:     Internal access within cluster
```

```
Backend ClusterIP:
  Name:      cloudroute-backend
  Type:      ClusterIP
  Port:      8000 → TargetPort 8000
  Selector:   app=cloudroute-backend
  Usage:     Internal API access
```

#### NodePort Services

```
Frontend NodePort:
  Name:      cloudroute-frontend-nodeport
  Type:      NodePort
  Port:      80 → TargetPort 3000
  NodePort:  30080
  Selector:   app=cloudroute-frontend
  Usage:     Quick external access for testing
```

```
Backend NodePort:
```

```
Name:          cloudroute-backend-nodeport
Type:          NodePort
Port:          8000 → TargetPort 8000
NodePort:      30081
Selector:      app=cloudroute-backend
Usage:         Quick external access for testing
```

## LoadBalancer Service (Optional for Lab)

```
External LB (MetalLB for bare-metal):
Name:          cloudroute-frontend-lb
Type:          LoadBalancer
Port:          80 → TargetPort 3000
Selector:      app=cloudroute-frontend
Usage:         Production-like external access
```

Note: For self-managed K8s on EC2, you can:

1. Use MetalLB for bare-metal LoadBalancer support
2. Or use NodePort with external proxy
3. Or use AWS NLB with cloud provider (if installed)

## ExternalName Services

```
External API Alias:
Name:          external-api
Type:          ExternalName
ExternalName:  jsonplaceholder.typicode.com
Usage:         Practice referencing external APIs

External Database Alias:
Name:          external-db
Type:          ExternalName
ExternalName:  mydb.example.com
Usage:         Practice referencing external databases
```

## Gateway API (Optional Advanced)

```
Gateway Class:  istio (or envoy-gateway)
Gateway Name:   cloudroute-gateway
Listeners:
- Name: http
```

```
Port: 80
Protocol: HTTP
```

```
HTTPRoute Name:    cloudroute-route
```

```
Rules:
```

- Path: /api → Backend Service (cloudroute-backend:8000)
- Path: / → Frontend Service (cloudroute-frontend:3000)

## Traffic Flow

```
External User
  ↓
Internet Gateway
  ↓
EC2 Public IP (NodePort or LoadBalancer)
  ↓
kube-proxy / iptables
  ↓
Service (ClusterIP/NodePort/LoadBalancer)
  ↓
Pod (Frontend or Backend)
```

## 5. Storage Plan

---

### EBS Volume Configuration

```
Master Root Volume:
```

```
Type:          gp3
Size:           20 GB
IOPS:           3000 (baseline)
Throughput:     125 MB/s
Encrypted:      Yes
Delete on Termination: Yes
```

```
Worker Root Volume:
```

```
Type:          gp3
Size:           20 GB
IOPS:           3000 (baseline)
Throughput:     125 MB/s
```



```
Encrypted:      Yes
Delete on Termination: Yes
```

## Additional EBS Volumes for Practice

Data Volume for StatefulSet Practice:

```
Type:          gp3
Size:          10 GB (additional)
IOPS:          3000
Throughput:    125 MB/s
Encrypted:     Yes
Availability:  us-east-1a
Attachment:    cloudroute-worker
Device Name:   /dev/sdf
```

Note: This volume can be used for:

- StatefulSet PVC practice
- Dynamic provisioning practice
- Persistent storage demos

## StorageClass Configuration

```
StorageClass Name:  cloudroute-efs-sc (or gp3-sc)
Provisioner:        kubernetes.io/aws-ebs
Parameters:
  type: gp3
  fsType: ext4
ReclaimPolicy:      Delete
VolumeBindingMode:  WaitForFirstConsumer
AllowVolumeExpansion: True
```

## CSI Driver Requirements

AWS EBS CSI Driver:

```
Image:      registry.k8s.io/sig-storage/csi-driver-aws-ebs:v1.27.0
Installation: kubectl apply -k "github.com/kubernetes-sigs/aws-ebs-csi-driver"
IAM Role:    Requires IAM role with EBS permissions
Usage:      Dynamic provisioning of EBS volumes
```

Note: For simple lab, you can use:

- Manual PV/PVC creation
- Or install CSI driver for dynamic provisioning

## Persistent Volume Claims (Practice)

### Frontend PVC:

```
Name:          cloudroute-frontend-data
Access Mode:    ReadWriteOnce
Storage:        1Gi
StorageClass:   cloudroute-efs-sc
```

### Backend PVC:

```
Name:          cloudroute-backend-data
Access Mode:    ReadWriteOnce
Storage:        1Gi
StorageClass:   cloudroute-efs-sc
```

### Database PVC (StatefulSet):

```
Name:          cloudroute-db-data
Access Mode:    ReadWriteOnce
Storage:        5Gi
StorageClass:   cloudroute-efs-sc
```

## 6. Container Registry Plan

---

### Recommendation: GitHub Container Registry (GHCR)

#### Why GHCR over ECR:

1. Free for public repositories
2. Integrated with GitHub Actions
3. No AWS billing for storage
4. Simple authentication with GITHUB\_TOKEN
5. Good for student labs

## Repository Naming

```
Registry:      ghcr.io
Organization:  <your-github-username>

Frontend Repo: ghcr.io/<your-username>/cloudroute-frontend
Backend Repo:  ghcr.io/<your-username>/cloudroute-backend
```

## Image Naming Structure

```
Frontend Images:
  Latest:      ghcr.io/<your-username>/cloudroute-frontend:latest
  SHA-based:   ghcr.io/<your-username>/cloudroute-frontend:<git-sha>
  Version:     ghcr.io/<your-username>/cloudroute-frontend:v1.0.0

Backend Images:
  Latest:      ghcr.io/<your-username>/cloudroute-backend:latest
  SHA-based:   ghcr.io/<your-username>/cloudroute-backend:<git-sha>
  Version:     ghcr.io/<your-username>/cloudroute-backend:v1.0.0
```

## Image Pull Policy

```
For Lab:
  imagePullPolicy: Always (for latest tags)
  imagePullPolicy: IfNotPresent (for SHA/version tags)

Image Pull Secret:
  Name:          ghcr-secret
  Type:          kubernetes.io/dockerconfigjson
  Registry:      ghcr.io
  Username:      <your-github-username>
  Password:      <github-personal-access-token>
```

## Alternative: AWS ECR (If Preferred)

```
ECR Repository Names:
  cloudroute-frontend
  cloudroute-backend

Lifecycle Policy:
  - Keep last 5 images
```

- Expire untagged images after 7 days

Cost: ~\$0.10/GB/month (minimal for lab)

## 7. CI/CD Readiness Plan

---

### GitHub Actions Configuration

```
Workflow File:    .github/workflows/deploy.yml
Trigger:          Push to main branch
Runner:           ubuntu-latest
```

### Required Secrets (Minimal)

#### Repository Secrets:

1. KUBE\_CONFIG → Base64-encoded kubeconfig for cluster access
2. REGISTRY\_URL → ghcr.io/<your-username>
3. AWS\_REGION → us-east-1 (if using ECR)

#### No Additional Variables Needed:

- Image names derived from repository name
- Tags derived from git SHA
- Namespace hardcoded as cloudroute-lab

### OIDC Alternative (Recommended)

```
OIDC Provider:    AWS IAM Identity Center (or GitHub OIDC)
Role:              GitHubActions-CloudRoute-Lab
Trust Policy:      GitHub OIDC federation
Permissions:
- ecr:GetAuthorizationToken
- ecr:BatchCheckLayerAvailability
- ecr:PutImage
- ecr:InitiateLayerUpload
- ecr:UploadLayerPart
- ecr:CompleteLayerUpload
```

**Benefits:**

- No long-lived AWS credentials
- Automatic credential rotation
- Better security posture

## Workflow Steps

1. Checkout code
2. Set up Docker Buildx
3. Login to container registry
4. Build Docker image
5. Push image with SHA tag
6. Update Kubernetes deployment
7. Verify rollout status

## Deployment Method

**Option A: kubectl set image**

- Simple, no additional tools
- Requires kubeconfig secret

**Option B: Helm upgrade**

- More complex, but declarative
- Requires Helm chart in repo

Recommendation for Lab: Option A (kubectl)

# 8. Monitoring Readiness Plan

---

## Prometheus Stack

```
Namespace:      monitoring
Prometheus:
  Image:         prom/prometheus:v2.49.0
  Storage:       5Gi PVC
  Retention:     7 days
Resources:
```

```
Requests:      200m CPU, 512Mi RAM
Limits:        500m CPU, 1Gi RAM

Grafana:
Image:         grafana/grafana:10.3.1
Storage:       2Gi PVC
Default Credentials:
Username:      admin
Password:      cloudroute-admin
Resources:
Requests:      100m CPU, 256Mi RAM
Limits:        200m CPU, 512Mi RAM
```

## Node Exporter (DaemonSet)

```
Name:          node-exporter
Image:          prom/node-exporter:v1.7.0
DaemonSet:      Yes (runs on every node)
Resources:
Requests:      50m CPU, 64Mi RAM
Limits:         100m CPU, 128Mi RAM
Ports:
- 9100 (metrics)

Purpose:        Collect node-level metrics
```

## Log Agent (DaemonSet)

```
Name:          log-agent
Image:          fluent/fluent-bit:3.0
DaemonSet:      Yes (runs on every node)
Resources:
Requests:      20m CPU, 32Mi RAM
Limits:         50m CPU, 64Mi RAM

Purpose:        Collect and forward logs
Output:         stdout (for lab) or Elasticsearch (optional)
```

## Resource Sizing for Small Cluster

#### Total Monitoring Overhead:

- Prometheus: ~1.5 GB RAM
- Grafana: ~0.5 GB RAM
- Node Exporter: ~0.2 GB RAM (both nodes)
- Log Agent: ~0.1 GB RAM (both nodes)
- Total: ~2.3 GB RAM

#### Available on 2x t3.medium (8 GB total):

- System/K8s: ~2 GB
- Monitoring: ~2.3 GB
- Applications: ~3.7 GB
- Buffer: ~0 GB (tight, but works for lab)

## Monitoring Endpoints

```
Prometheus UI:    http://<master-ip>:30090
Grafana UI:       http://<master-ip>:30030
Node Exporter:    http://<node-ip>:9100/metrics
```

#### Service Types:

- Prometheus: NodePort (30090)
- Grafana: NodePort (30030)

## 9. Security Plan

---

### IAM Roles

#### Master Node Role

```
Role Name:        cloudroute-master-role
Trust Policy:      EC2 Service
Managed Policies:
  - AmazonEKS_CNI_Policy (optional, for VPC CNI)
  - AmazonEC2ContainerRegistryReadOnly

Custom Inline Policy:
{
  "Version": "2012-10-17",
```

```
"Statement": [  
  {  
    "Effect": "Allow",  
    "Action": [  
      "ec2:DescribeInstances",  
      "ec2:DescribeVolumes",  
      "ec2:AttachVolume",  
      "ec2:DetachVolume"  
    ],  
    "Resource": "*"   
  }  
]
```

Purpose: Allow EC2 interactions for K8s cloud provider

## Worker Node Role

Role Name: cloudroute-worker-role  
Trust Policy: EC2 Service  
Managed Policies:  
- AmazonEC2ContainerRegistryReadOnly

Purpose: Pull images from ECR (if using ECR)

## Least Privilege Recommendations

1. SSH Access:
  - Restrict to your IP only (not 0.0.0.0/0)
  - Use key-based authentication only
  - Disable root login
2. Kubernetes Access:
  - Use RBAC for fine-grained access
  - Create service accounts for applications
  - Limit namespace access
3. Container Registry:
  - Use read-only tokens for image pulls
  - Rotate credentials regularly
4. Network:
  - Restrict NodePort access to known IPs



- Use NetworkPolicies for pod-to-pod traffic
- Enable VPC Flow Logs for debugging

## Security Group Rules (Summary)

### Master SG:

#### Inbound:

- 22 (SSH) → Your IP only
- 6443 (API) → Worker SG + Your IP
- 2379-2380 (etcd) → Master SG only
- 10250 (kubelet) → Master + Worker SG
- 80, 443 → 0.0.0.0/0 (for web apps)
- 30000-32767 → 0.0.0.0/0 (NodePort)

### Worker SG:

#### Inbound:

- 22 (SSH) → Your IP only
- 10250 (kubelet) → Master SG only
- 80, 443 → 0.0.0.0/0
- 30000-32767 → 0.0.0.0/0 (NodePort)

## SSH Access Plan

### SSH Configuration:

- User: ubuntu (for Ubuntu AMI)
- Key: cloudroute-keypair.pem
- Port: 22
- Authentication: Key-based only
- Root Login: Disabled

### SSH Commands:

- Master: `ssh -i cloudroute-keypair.pem ubuntu@<master-public-ip>`
- Worker: `ssh -i cloudroute-keypair.pem ubuntu@<worker-public-ip>`

### Jump Host (Optional):

- SSH into master, then ssh into worker
- Or use ProxyJump for direct access

## Kubernetes Access Plan

**kubeconfig Location:**

Master:     /etc/kubernetes/admin.conf  
 Local:     ~/.kube/config

**Access Methods:**

1. kubectl (from master node)
2. kubectl (from local machine with kubeconfig)
3. Kubernetes Dashboard (optional)

**RBAC Setup:**

- Create admin role for yourself
- Create limited roles for applications
- Use ServiceAccounts for pods

## 10. Final Output Format

---

### Infrastructure Architecture Diagram

[Internet]

↓

[Internet Gateway] → cloudroute-igw

↓

[VPC] → cloudroute-vpc (10.0.0.0/16)

↓

[Public Subnet] → cloudroute-public-subnet-1a (10.0.1.0/24)

↓

| [Master Node]           | [Worker Node]     |
|-------------------------|-------------------|
| cloudroute-master       | cloudroute-worker |
| t3.medium               | t3.medium         |
| 20 GB root              | 20 GB root        |
| + 10 GB data (optional) |                   |
| Components:             | Components:       |
| - kube-apiserver        | - kubelet         |
| - kube-scheduler        | - kube-proxy      |

|                       |                 |
|-----------------------|-----------------|
| - kube-controller-mgr | - Calico node   |
| - etcd                | - Node Exporter |
| - kubelet             | - Log Agent     |
| - kube-proxy          |                 |
| - Calico node         |                 |
| - Prometheus          |                 |
| - Grafana             |                 |
| - Node Exporter       |                 |
| - Log Agent           |                 |

## Resource List

### AWS Resources

1. VPC: clouddroute-vpc
2. Subnet: clouddroute-public-subnet-1a
3. Internet Gateway: clouddroute-igw
4. Route Table: clouddroute-public-rt
5. Security Group (Master): clouddroute-master-sg
6. Security Group (Worker): clouddroute-worker-sg
7. EC2 Instance (Master): clouddroute-master
8. EC2 Instance (Worker): clouddroute-worker
9. EBS Volume (Master): clouddroute-master-root (20 GB)
10. EBS Volume (Worker): clouddroute-worker-root (20 GB)
11. EBS Volume (Data): clouddroute-worker-data (10 GB, optional)
12. Key Pair: clouddroute-keypair
13. IAM Role (Master): clouddroute-master-role
14. IAM Role (Worker): clouddroute-worker-role

### Kubernetes Resources (to be created)

1. Namespace: clouddroute-lab
2. Namespace: monitoring
3. Deployment: clouddroute-frontend
4. Deployment: clouddroute-backend
5. DaemonSet: node-exporter
6. DaemonSet: log-agent
7. StatefulSet: clouddroute-db (optional)
8. Service: clouddroute-frontend (ClusterIP)

- |                                                              |                                                |
|--------------------------------------------------------------|------------------------------------------------|
| 9. Service: cloudroute-backend (ClusterIP)                   | 19. ConfigMap: cloudroute-config               |
| 10. Service: cloudroute-frontend-nodeport (NodePort)         | 20. Secret: ghcr-secret                        |
| 11. Service: cloudroute-backend-nodeport (NodePort)          | 21. PVC: prometheus-data                       |
| 12. Service: cloudroute-frontend-lb (LoadBalancer, optional) | 22. PVC: grafana-data                          |
| 13. Service: external-api (ExternalName)                     | 23. PVC: cloudroute-frontend-data              |
| 14. Service: external-db (ExternalName)                      | 24. PVC: cloudroute-backend-data               |
| 15. Service: prometheus (NodePort)                           | 25. PVC: cloudroute-db-data                    |
| 16. Service: grafana (NodePort)                              | 26. StorageClass: cloudroute-efs-sc            |
| 17. Gateway: cloudroute-gateway (optional)                   | 27. ServiceAccount: cloudroute-app             |
| 18. HTTPRoute: cloudroute-route (optional)                   | 28. ClusterRole: cloudroute-app-role           |
|                                                              | 29. ClusterRoleBinding: cloudroute-app-binding |

## Recommended Names for Every Resource

### AWS Resource Names

```
cloudroute-vpc
cloudroute-public-subnet-1a
cloudroute-igw
cloudroute-public-rt
cloudroute-master-sg
cloudroute-worker-sg
cloudroute-master
cloudroute-worker
cloudroute-master-root
cloudroute-worker-root
cloudroute-worker-data
cloudroute-keypair
cloudroute-master-role
cloudroute-worker-role
```

### Kubernetes Resource Names

```
cloudroute-lab (namespace)
monitoring (namespace)
cloudroute-frontend (deployment)
cloudroute-backend (deployment)
node-exporter (daemonset)
log-agent (daemonset)
cloudroute-db (statefulset)
cloudroute-frontend (service)
cloudroute-backend (service)
cloudroute-frontend-nodeport (service)
cloudroute-backend-nodeport (service)
cloudroute-frontend-lb (service)
external-api (service)
external-db (service)
prometheus (service)
grafana (service)
cloudroute-gateway (gateway)
cloudroute-route (httproute)
cloudroute-config (configmap)
ghcr-secret (secret)
prometheus-data (pvc)
grafana-data (pvc)
cloudroute-frontend-data (pvc)
cloudroute-backend-data (pvc)
cloudroute-db-data (pvc)
cloudroute-efs-sc (storageclass)
cloudroute-app (serviceaccount)
cloudroute-app-role (clusterrole)
cloudroute-app-binding (clusterrolebinding)
```

## Minimal Variables

### Environment Variables

```
# Application
APP_ENV=production
ALLOWED_ORIGINS=http://<master-ip>:30080
EXTERNAL_SERVICE_ALIAS=external-api.cloudroute-lab.svc.cluster.local

# Container Registry
REGISTRY_URL=ghcr.io/<your-username>
IMAGE_TAG=latest

# Kubernetes
```

```
NAMESPACE=cloudroute-lab  
MASTER_IP=<master-public-ip>  
WORKER_IP=<worker-public-ip>
```

## GitHub Secrets

```
KUBE_CONFIG      → Base64-encoded kubeconfig  
REGISTRY_URL     → ghcr.io/<your-username>  
AWS_REGION      → us-east-1 (if using ECR)
```

## GitHub Variables

None required for basic setup

## Deployment Order

### Phase 1: AWS Infrastructure (InfraPilot)

1. Create VPC and Subnet
2. Create Internet Gateway and Route Table
3. Create Security Groups
4. Create Key Pair
5. Create IAM Roles
6. Create EC2 Instances (Master + Worker)
7. Create EBS Volumes
8. Attach Volumes to Instances
9. Configure SSH Access
10. Verify Network Connectivity

### Phase 2: Kubernetes Cluster (Manual or Automated)

1. SSH into Master Node
2. Install containerd on both nodes
3. Install kubeadm, kubelet, kubectl on both nodes
4. Initialize Master with kubeadm init
5. Copy kubeconfig to local machine
6. Install Calico CNI on Master
7. Generate join command for Worker

8. SSH into Worker and run join command
9. Verify cluster with `kubectl get nodes`
10. Label Worker node

### Phase 3: Storage Setup (Manual)

1. Create StorageClass
2. Create PVCs for monitoring
3. Create PVCs for applications
4. (Optional) Attach additional EBS volume
5. (Optional) Install EBS CSI Driver

### Phase 4: Monitoring Stack (Manual)

1. Create monitoring namespace
2. Deploy Prometheus
3. Deploy Grafana
4. Deploy Node Exporter DaemonSet
5. Deploy Log Agent DaemonSet
6. Verify monitoring endpoints

### Phase 5: Application Deployment (Manual)

1. Create `clouddroute-lab` namespace
2. Create ConfigMap and Secrets
3. Deploy Frontend
4. Deploy Backend
5. Create ClusterIP Services
6. Create NodePort Services
7. (Optional) Create LoadBalancer Service
8. Create ExternalName Services
9. (Optional) Install Gateway API controller
10. (Optional) Create Gateway and HTTPRoute

### Phase 6: CI/CD Setup (Manual)

1. Create GitHub Actions workflow
2. Add repository secrets
3. Test build and push

4. Test deployment
5. Verify full pipeline

## What InfraPilot Should Create Automatically

### AWS Resources (Terraform/CloudFormation)

1. VPC with DNS support
2. Public Subnet with auto-assign public IP
3. Internet Gateway attached to VPC
4. Route Table with internet route
5. Security Groups with rules
6. EC2 Instances with user data
7. EBS Volumes (root + optional data)
8. Key Pair (or import existing)
9. IAM Roles with policies
10. Elastic IP (optional)

### Kubernetes Resources (Helm Charts/Manifests)

1. Namespaces (cloudroute-lab, monitoring)
2. StorageClass
3. Monitoring stack (Prometheus, Grafana, Node Exporter, Log Agent)
4. Application deployments (frontend, backend)
5. Services (ClusterIP, NodePort, LoadBalancer, ExternalName)
6. ConfigMaps and Secrets
7. PVCs for persistent storage
8. RBAC resources (ServiceAccount, ClusterRole, ClusterRoleBinding)

### CI/CD Resources (GitHub Actions)

1. Workflow file (.github/workflows/deploy.yml)
2. Dockerfile for frontend
3. Dockerfile for backend
4. .dockerignore files
5. README with deployment instructions



## Cost Estimation

### Monthly Cost Breakdown (On-Demand):

- EC2 Instances: ~\$60/month
- EBS Volumes: ~\$4/month
- Data Transfer: ~\$5/month
- **Total: ~\$69/month**

### Cost Optimization:

- Use Spot Instances: ~\$18/month (60% savings)
- Use Reserved Instances: ~\$38/month (45% savings)

## Document Information

---

Created:

2026-07-08

Project:

CloudRoute Lab

Version:

1.0

Author:

InfraPilot

Purpose:

Complete infrastructure blueprint for student Kubernetes lab